

第十一章：网络协议栈

一、核心概念

1.1 TCP/IP 分层模型

应用层	HTTP, DNS, SSH, ...	用户空间
传输层	TCP (可靠) / UDP (不可靠)	内核
网络层	IP (路由, 分片)	内核
链路层	Ethernet (MAC 地址, ARP)	驱动
物理层	网卡硬件	硬件

1.2 Socket API

BSD Socket 是网络编程的标准接口：

函数	作用
<code>socket(domain, type, protocol)</code>	创建套接字
<code>bind(fd, addr, len)</code>	绑定地址
<code>listen(fd, backlog)</code>	监听 (TCP 服务端)
<code>connect(fd, addr, len)</code>	连接 (TCP 客户端)
<code>accept(fd, addr, len)</code>	接受连接 (TCP 服务端)
<code>sendto(fd, buf, len, flags, addr, addrlen)</code>	发送 (UDP)
<code>recvfrom(fd, buf, len, flags, addr, addrlen)</code>	接收 (UDP)
<code>close(fd)</code>	关闭

1.3 TCP vs UDP

特性	TCP	UDP
可靠性	保证 (重传、确认、排序)	不保证
连接性	面向连接 (三次握手)	无连接
流控制	滑动窗口	无
开销	较高 (头部 20B, 状态维护)	低 (头部 8B)
适用	文件传输、Web	DNS、视频流、游戏

1.4 以太网帧格式

--

```
[目的MAC 6B][源MAC 6B][类型 2B][数据 46-1500B][FCS 4B]
类型: 0x0800=IP, 0x0806=ARP
```

1.5 VirtIO 网卡

与 VirtIO 块设备类似，但有两个 virtqueue：

- **RX 队列**：预先放满空缓冲区，Host 填入收到的帧
- **TX 队列**：Guest 放待发帧，Host 取走发出

二、基本推论

推论 1：协议栈是分层解耦的典范。 每层只关心自己的头部和逻辑，通过封装/解封装与上下层交互。这让替换某一层的实现（如换 WiFi 链路层）不影响上层。

推论 2：TCP 的复杂性来自可靠传输。 重传超时、快速重传、拥塞控制（慢启动/拥塞避免/快速恢复）、窗口管理——这些让 TCP 的实现远比 UDP 复杂。

推论 3：网络收包是中断驱动的。 网卡收到帧 → 中断 → 驱动从 RX 队列取帧 → 传给协议栈。高吞吐场景下用 NAPI（轮询+中断混合）减少中断风暴。

三、Linux / 通用实现

3.1 Linux 网络栈

```
用户空间: socket API
          ↓ syscall
内核:
sock 层:   struct socket → struct sock
传输层:   tcp_sendmsg / udp_sendmsg
网络层:   ip_output → 路由查找 → ip_local_deliver
链路层:   dev_queue_xmit → netdev_start_xmit
驱动:     ndo_start_xmit → 硬件队列
```

NAPI (New API)：高吞吐时从纯中断切到轮询模式：

```
中断来 → 关闭该中断源 → 启动 NAPI poll
poll: 批量处理 RX 队列中的包（最多 budget 个）
处理完 → 重新开启中断
```

3.2 Linux 的 sk_buff

sk_buff 是 Linux 网络数据包的核心数据结构：

```
struct sk_buff {
    unsigned char *head, *data, *tail, *end;
    // data 指向当前层的头部
    // 向下传递时 push (添加头部)
    // 向上传递时 pull (剥离头部)
};
```

3.3 select / poll / epoll

机制	复杂度	特点
select	O(n)	fd 数量限制 1024
poll	O(n)	无 fd 限制，接口更好
epoll	O(1)	事件驱动，高性能，Linux 特有

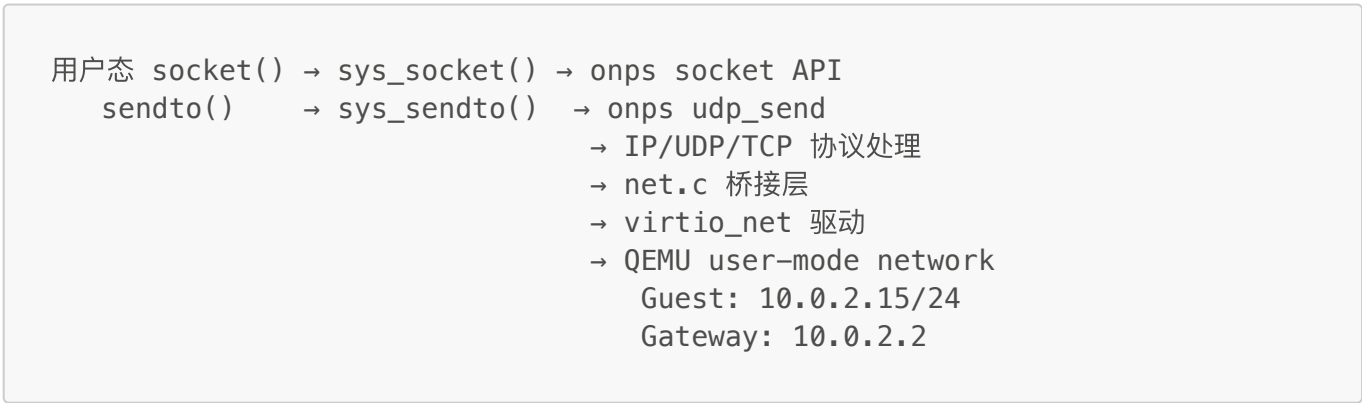
epoll 的核心：内核维护一个红黑树存放关注的 fd，事件发生时将 fd 加入就绪链表。epoll_wait 只返回就绪的 fd。

四、RUOS 的实现

核心文件

- src/network/net.c — 网络初始化和桥接
- src/network/open-npstack/ — onps TCP/IP 栈
- src/syscall/sysnet.c — socket 系统调用

4.1 架构



4.2 onps 适配

为 onps 实现 OS 适配层：

onps 需要	RUOS 实现
mutex	spinlock
semaphore	spinlock + 计数器 + sleep/wakeup

onps 需要	RUOS 实现
sleep(ms)	ticks 转换 + sleep channel
内存分配	kalloc/kfree

收包用内核线程：

```
kthread_create(ethernet_ii_recv, NULL, "net_rx");
// 无限循环：virtio_net_recv() → 解析以太网帧 → 交给协议栈
```

4.3 Socket 集成到文件系统

新增 `FD_SOCKET` 文件类型。`sys_socket()` 创建 onps socket 后包装为 `struct file`。关闭时走 `onps_close()`。

五、八股 / 面试高频问题

Q: TCP 三次握手的过程？为什么不是两次？ A: SYN → SYN+ACK → ACK。两次的话，服务端无法确认客户端收到了 SYN+ACK，可能导致历史连接被误接受。三次确保双方都确认了对方的接收能力。

Q: TCP 四次挥手？为什么需要 TIME_WAIT？ A: FIN → ACK → FIN → ACK。TIME_WAIT 等待 2MSL，确保最后一个 ACK 到达对方，且网络中旧连接的包消失。

Q: TCP 拥塞控制？ A: 慢启动（指数增长）→ 拥塞避免（线性增长）→ 丢包后快速重传/快速恢复。算法：Reno, CUBIC（Linux 默认），BBR（Google）。

Q: epoll 和 select 的区别？为什么 epoll 更快？ A: select 每次调用需要拷贝 fd 集合到内核并遍历。epoll 在内核维护红黑树，只返回就绪 fd，避免了 O(n) 遍历。适合大量连接但少数活跃的场景。

Q: 什么是零拷贝（Zero-Copy）？ A: 避免数据在用户空间和内核空间之间拷贝。Linux 的 `sendfile()` 直接将文件数据从 page cache 发到 socket 缓冲区，跳过用户空间拷贝。`mmap() + write()` 也是一种零拷贝思路。

六、项目贡献亮点

- 嵌入 onps TCP/IP 栈并实现 OS 适配层（mutex/semaphore/sleep 映射到内核原语）
- 实现 VirtIO 网卡驱动（双队列 RX/TX），理解网络设备的中断驱动收包模型
- 实现 BSD socket API 系统调用，集成到内核文件描述符体系

七、已知限制与设计取舍

限制	原因	Linux 对照
无 select/poll/epoll	未实现 I/O 多路复用	epoll 是核心
无 NAPI	中断处理在中断上下文	NAPI 减少中断风暴

限制	原因	Linux 对照
无 ICMP	onps 不支持 ping	Linux 内核原生支持
静态 IP	无 DHCP	Linux 有 dhclient
onps 资源有限	并发连接数少	Linux 无此限制